

Week 5 Lab: Sensors & ADC Applications

EE0827 Embedded System Applications

Overview

In this lab, you will interface two analog sensors with the STM32 ADC: a K-type thermocouple with an LM741 amplifier circuit and an LDR (Light Dependent Resistor). You will null the amplifier offset, implement digital filtering, and optionally perform 2-point calibration to convert ADC readings to temperature.

Time allocation: 90 minutes

- Part 1 - CubeMX Configuration: 15 min
- Part 2 - Thermocouple + Amplifier: 30 min
- Part 3 - Light Sensor: 20 min
- Part 4 - Digital Filtering: 15 min
- Part 5 - Two-Point Calibration (*optional*): 10 min

Learning Objectives

By the end of this lab, you will be able to:

1. Configure the STM32 ADC for multi-channel operation
2. Interface a thermocouple sensor with external amplification to the ADC
3. Design and implement voltage dividers for resistive sensors (LDR)
4. Implement a moving average filter in firmware
5. (*Optional*) Perform 2-point calibration and apply calibration coefficients

Safety Gate

Safety Checklist (Complete Before Powering On)

- ☐ Bench supply set to $\pm 12\text{V}$ (series mode) before powering on the LM741
- ☐ Thermocouple center wire to pin 3 (+), brass body to GND
- ☐ No shorts between breadboard rows
- ☐ USB cable connected for programming and UART output

Sensor Handling

1. **Verify amplifier output with multimeter** before connecting to STM32 PA0 — must be below 3.3V
2. **Keep analog wires short** — thermocouple signals are in the μV range and pick up noise easily
3. **Do not touch the thermocouple tip** during measurements — body heat will shift the reading

Required Parts and Tools

Hardware

- STM32 Nucleo-F401RE board
- Breadboard and jumper wires (~ 10)
- $10\text{k}\Omega$ resistor (for LDR voltage divider)
- K-type thermocouple probe (stove/oven type)
- LM741 op-amp (DIP-8)
- $220\text{k}\Omega$ resistor (R_f), $1\text{k}\Omega$ resistors $\times 2$ (R_g and R_p)
- $5\text{--}10\text{k}\Omega$ potentiometer (offset null)
- Bench power supply ($\pm 12\text{V}$, series mode)
- LDR photoresistor ($1\text{k}\Omega\text{--}100\text{k}\Omega$ range)
- USB cable (programming and UART)

Tools

- Multimeter (voltage verification)
- Oscilloscope (noise measurement)
- Serial terminal (PuTTY, TeraTerm, or CubeIDE terminal)

! PRE-LAB SECTION

Complete the following **BEFORE** arriving at the lab session.

Pre-Lab Checklist

Complete before arriving at the lab:

- ☐ Read theory slides on ADC operation and sensor interfacing
- ☐ Understand how a thermocouple generates voltage ($\sim 41\mu\text{V}/^\circ\text{C}$ for K-type)
- ☐ Understand why external amplification is needed for thermocouple signals
- ☐ Understand voltage divider calculations
- ☐ Complete all pre-lab calculations below

Pre-Lab Questions

1. A K-type thermocouple outputs $\sim 41\mu\text{V}/^\circ\text{C}$. What is the ADC's voltage resolution at 12-bit / 3.3V? Why can't the STM32 read the thermocouple directly?

2. Why do resistive sensors (like LDR) need a voltage divider while voltage-output sensors (like the amplifier) don't? _____

3. What is the purpose of a moving average filter in sensor applications? _____

Pre-Lab Calculations

1. Thermocouple + Amplifier Output

A K-type thermocouple outputs approximately $41\mu\text{V}$ per degree Celsius. The LM741 non-inverting amplifier has gain $A_v = 1 + R_f/R_g = 1 + 220\text{k}/1\text{k} = 221$.

At room temperature (25°C), what is the raw thermocouple voltage?

$$V_{TC} = 25^\circ\text{C} \times 41\mu\text{V}/^\circ\text{C} = \text{_____}m\text{V}$$

The STM32 ADC resolution is $3.3V/4095 \approx 0.806\text{ mV}$. What minimum amplifier gain is needed to produce at least 1 LSB per °C?

$$A_{v,min} = \frac{0.806\text{ mV}}{41\text{ }\mu\text{V}} = \underline{\hspace{2cm}}$$

With $A_v = 221$, what amplifier output voltage do you expect at 25°C?

$$V_{out} = V_{TC} \times A_v = \underline{\hspace{2cm}}\text{ mV}$$

What ADC value would that correspond to? ($V_{REF} = 3.3V$, 12-bit)

$$ADC = \frac{V_{out}}{3.3V} \times 4095 = \underline{\hspace{2cm}}$$

2. LDR Voltage Divider

Using a $10\text{ k}\Omega$ resistor as R_{fixed} (top of divider) with LDR on bottom:

$$V_{out} = V_{cc} \times \frac{R_{LDR}}{R_{fixed} + R_{LDR}}$$

Calculate V_{out} for the following LDR resistance values:

Light Condition	R_{LDR}	V_{out} (calculated)
Bright	$1\text{ k}\Omega$	<u> </u> V
Room light	$10\text{ k}\Omega$	<u> </u> V
Dark	$100\text{ k}\Omega$	<u> </u> V

3. Filter Design

For 10 Hz sampling with 8-sample moving average, how many seconds for full response?

$$t_{response} = \frac{8\text{ samples}}{10\text{ Hz}} = \underline{\hspace{2cm}}\text{ s}$$

IN-LAB SECTION

The following sections are completed during the lab session.

Part 1

CubeMX ADC Configuration

15 minutes

Step 1.1: Create New Project

1. Open STM32CubeIDE
2. Create new STM32 project for Nucleo-F401RE
3. Name it W05_Sensors_ADC

Step 1.2: Configure ADC Channels

In the Pinout view:

1. Click on **PA0** → Select **ADC1_IN0** (Temperature sensor)
2. Click on **PA1** → Select **ADC1_IN1** (Light sensor)

Step 1.3: Configure ADC Parameters

Navigate to **Analog** → **ADC1** in the left panel:

Parameter	Value
Clock Prescaler	PCLK2 divided by 4
Resolution	12 bits
Data Alignment	Right
Scan Conversion Mode	Enabled
Number of Conversion	2

Rank 1: Channel 0, Sampling Time 84 Cycles

Rank 2: Channel 1, Sampling Time 84 Cycles

Step 1.4: Configure UART

Navigate to **Connectivity** → **USART2**:

Parameter	Value
Mode	Asynchronous
Baud Rate	115200

Step 1.5: Generate Code

1. Save the IOC file
2. Click **Project** → **Generate Code**

Checkpoint 1

Verify your CubeMX configuration before proceeding.

Part 2

Thermocouple + Amplifier Interface

25 minutes

Step 2.1: Hardware Setup

Wire the K-type thermocouple through the LM741 non-inverting amplifier to the STM32:

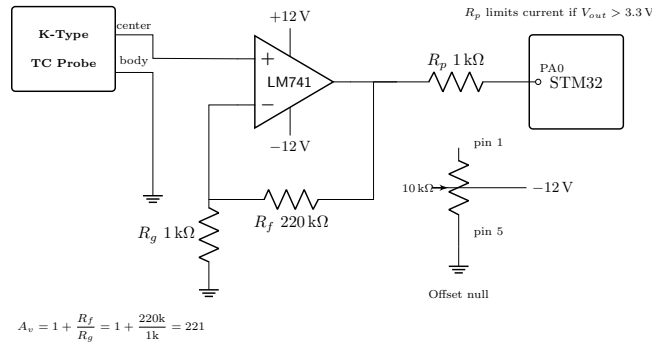


Figure 1: LM741 thermocouple amplifier circuit

The stove thermocouple is a coaxial probe — the two dissimilar metals share a single cable. The **center conductor** is one lead and the **brass body/housing** is the other.

Connection	Details
TC center wire	LM741 pin 3 (non-inverting input, +)
TC brass body	Circuit GND
R_f (220kΩ)	Pin 6 (output) to pin 2 (inverting input, −)
R_g (1kΩ)	Pin 2 to GND
Pin 7 (V+)	+12V from bench supply
Pin 4 (V−)	−12V from bench supply
R_p (1kΩ)	Pin 6 to PA0 (output protection)
Offset null pot (5–10kΩ)	Between pins 1 and 5, wiper to −12V

Verify Before Connecting to MCU

Power the amplifier and measure pin 6 (output) with a multimeter **before** connecting R_p to PA0. The output must be below 3.3 V at room temperature. If not, check wiring or reduce gain.

Step 2.2: Null Offset and Verify

1. Set bench supply to $\pm 12\text{V}$ (series mode, both channels at 12V)
2. **Short the thermocouple input** — connect center wire to brass body (forces 0mV)
3. Measure LM741 output (pin 6) with a multimeter
4. Turn the offset null pot until output reads as close to **0.000V** as possible
5. Remove the short, reconnect the thermocouple
6. Warm the thermocouple tip with your fingers — output should increase
7. **Once confirmed $< 3.3\text{V}$** , connect R_p output to PA0

Measurement	Value
Output after offset null (shorted input)	_____ mV
Output at room temp	_____ mV
Output with finger heat	_____ mV

Why offset null matters

The LM741 has an inherent input offset voltage of 1–6 mV. At a gain of ~ 220 , this becomes 220–1320 mV at the output — comparable to the thermocouple signal itself. The offset null pot cancels this error *before* amplification.

Step 2.3: Basic ADC Reading Code

Add to `main.c` (in USER CODE sections):

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include <string.h>
/* USER CODE END Includes */

/* USER CODE BEGIN PV */
uint16_t adc_temp_raw = 0;
float temp_voltage = 0;
char uart_buf[64];
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
uint16_t read_adc_channel(uint8_t channel) {
    ADC_ChannelConfTypeDef sConfig = {0};
    sConfig.Channel = (channel == 0) ? ADC_CHANNEL_0 : ADC_CHANNEL_1;
```



```

    sConfig.Rank = 1;
    sConfig.SamplingTime = ADC_SAMPLETIME_84CYCLES;

    HAL_ADC_ConfigChannel(&hadc1, &sConfig);
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, 10);
    return HAL_ADC_GetValue(&hadc1);
}
/* USER CODE END 0 */

```

Step 2.4: Main Loop Code

Although the gain is known ($A_v \approx 221$), the cold junction offset and component tolerances make a direct voltage-to-temperature formula unreliable. We display raw ADC and voltage here; calibrated temperature conversion is covered in Part 5.

```

while (1) {
    adc_temp_raw = read_adc_channel(0);
    temp_voltage = (adc_temp_raw / 4095.0f) * 3.3f;

    sprintf(uart_buf, "TC: ADC=%4d, V=%.3fV\r\n",
            adc_temp_raw, temp_voltage);
    HAL_UART_Transmit(&huart2, (uint8_t *)uart_buf,
                      strlen(uart_buf), HAL_MAX_DELAY);
    HAL_Delay(500);
}

```

Step 2.5: Test Thermocouple Response

1. Build and flash the project
2. Open serial terminal (115200 baud)
3. Pinch the thermocouple tip — ADC value should increase
4. Release and blow on it — ADC value should decrease

Measurement	Value
ADC raw at room temp	_____
ADC raw with finger heat	_____

Checkpoint 2

Verify that the ADC value changes when you heat/cool the thermocouple tip.

Part 3

Light Sensor Interface

20 minutes

Step 3.1: Hardware Setup

Build the LDR voltage divider circuit:

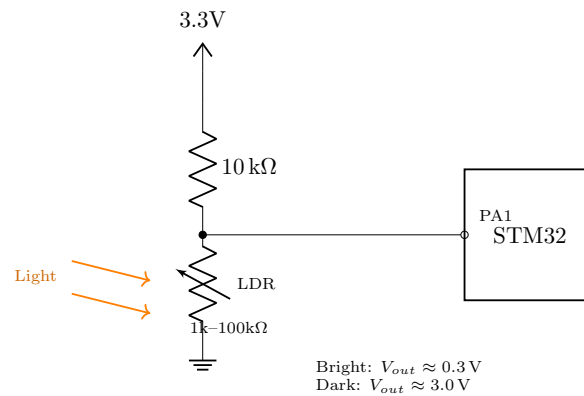


Figure 2: LDR voltage divider interface

Step 3.2: Verify Hardware

Measure voltage at PA1 junction with multimeter:

Condition	Expected	Measured
Bright light	~0.3V	_____ V
Room light	~1.65V	_____ V
Dark (covered)	~3.0V	_____ V

Step 3.3: Add Light Sensor Code

Add variables and update main loop:

```
/* USER CODE BEGIN PV */  
uint16_t adc_light_raw = 0;  
float light_voltage = 0;  
float light_percent = 0;  
/* USER CODE END PV */
```

```

while (1) {
    // Thermocouple (raw voltage - calibration comes in Part 4)
    adc_temp_raw = read_adc_channel(0);
    temp_voltage = (adc_temp_raw / 4095.0f) * 3.3f;

    // Light (invert: 0V=bright, 3.3V=dark)
    adc_light_raw = read_adc_channel(1);
    light_voltage = (adc_light_raw / 4095.0f) * 3.3f;
    light_percent = 100.0f - (light_voltage / 3.3f * 100.0f);

    sprintf(uart_buf, "TC: %.3fV | Light: %.0f%%\r\n",
            temp_voltage, light_percent);
    HAL_UART_Transmit(&huart2, (uint8_t *)uart_buf,
                      strlen(uart_buf), HAL_MAX_DELAY);
    HAL_Delay(500);
}

```

Checkpoint 3

Verify both sensors are working before proceeding.

Step 4.1: Implement Moving Average Filter

```
/* USER CODE BEGIN PV */
#define FILTER_SIZE 8
uint16_t temp_buffer[FILTER_SIZE] = {0};
uint16_t light_buffer[FILTER_SIZE] = {0};
uint8_t filter_index = 0;
uint32_t temp_sum = 0, light_sum = 0;
/* USER CODE END PV */

uint16_t filter_update(uint16_t *buffer, uint32_t *sum, uint16_t sample) {
    *sum -= buffer[filter_index];
    buffer[filter_index] = sample;
    *sum += sample;
    return *sum / FILTER_SIZE;
}
```

Step 4.2: Update Main Loop with Filtering

```
while (1) {
    adc_temp_raw = read_adc_channel(0);
    uint16_t temp_filt = filter_update(temp_buffer, &temp_sum, adc_temp_raw);

    adc_light_raw = read_adc_channel(1);
    uint16_t light_filt = filter_update(light_buffer, &light_sum, adc_light_raw);

    filter_index = (filter_index + 1) % FILTER_SIZE;

    sprintf(uart_buf, "Raw: T=%4d L=%4d | Filt: T=%4d L=%4d\r\n",
            adc_temp_raw, adc_light_raw, temp_filt, light_filt);
    HAL_UART_Transmit(&huart2, (uint8_t *)uart_buf,
                      strlen(uart_buf), HAL_MAX_DELAY);
    HAL_Delay(100);
}
```

Step 4.3: Observe Filter Behavior

1. Touch LDR wire briefly — observe raw spikes, filtered stays smooth
2. Blow on thermocouple — observe gradual filtered response

Checkpoint 4

Verify filtering is working — raw values should jitter while filtered values stay smooth.

Part 5

Two-Point Calibration

10 minutes (optional)

To convert ADC counts into actual temperature, you need two known reference points. The calibration compensates for the amplifier gain, offset, and cold junction effects all at once.

Step 5.1: Collect Calibration Data

Record the filtered ADC value at two known temperatures:

Point	Method	Known Temp	Filtered ADC
1	Room temperature (read from thermometer or phone)	_____ °C	_____
2	Warm water or sustained finger contact	_____ °C	_____

Cold Junction Compensation

A thermocouple measures the *difference* between the hot junction (probe tip) and cold junction (where the wires meet the copper traces). Your 2-point calibration implicitly compensates for the cold junction offset at the current ambient temperature.

Step 5.2: Add Calibration to Code

```
/* USER CODE BEGIN PV */
// Fill in your measured calibration values:
float cal_adc_1 = 0;    // ADC at reference point 1 (room temp)
float cal_temp_1 = 0;   // Known temp at point 1 (°C)
float cal_adc_2 = 0;    // ADC at reference point 2 (warm)
float cal_temp_2 = 0;   // Known temp at point 2 (°C)
/* USER CODE END PV */

// In main loop, after filtering:
float temp_celsius = cal_temp_1
    + (temp_filt - cal_adc_1)
```

```

    * (cal_temp_2 - cal_temp_1)
    / (cal_adc_2 - cal_adc_1);

sprintf(uart_buf, "Temp: %.1f C (filt ADC=%d)\r\n",
        temp_celsius, temp_filt);
HAL_UART_Transmit(&huart2, (uint8_t *)uart_buf,
                  strlen(uart_buf), HAL_MAX_DELAY);

```

Step 5.3: Verify Calibration

1. Fill in your two calibration values and rebuild
2. Check that room temperature reads correctly on the serial terminal
3. Warm the probe — temperature should increase smoothly

Checkpoint 5 (optional)

Verify calibrated temperature output is reasonable at room temperature.

Measurement Summary

ID	Measurement	Expected	Measured	Pass
M1	Amplifier output after offset null (shorted)	~0mV		
M2	Amplifier output at room temp	100–500mV		
M3	ADC responds to finger heat	Increase visible		
M4	Filter smooths raw jitter	Filt < Raw variation		

Analysis Questions

1. Why does the LDR need a voltage divider but the thermocouple amplifier doesn't? _____
2. How would you modify the filter to respond faster? What's the trade-off? _
3. What happens to your calibration if the room temperature changes signifi-

cantly? (Hint: think about cold junction compensation.) _____

Completion Checklist

- ☐ Checkpoint 1: CubeMX configuration verified
- ☐ Checkpoint 2: Thermocouple + amplifier working
- ☐ Checkpoint 3: Both sensors working
- ☐ Checkpoint 4: Filtering demonstrated
- ☐ Checkpoint 5 (*optional*): Calibrated temperature output verified
- ☐ All measurements recorded
- ☐ Analysis questions answered

Success Criteria

This lab is ungraded practice. Use these criteria to gauge your progress:

Area	What to Aim For
Pre-lab	All calculations completed before lab
ADC Configuration	CubeMX settings correct, code compiles
Thermocouple + Amplifier	Offset nulled, ADC responds to temperature changes
Light Sensor	Readings change with light level
Filtering	Smooth output, reduced noise visible
Calibration (<i>optional</i>)	Temperature reads correctly at room temp
Analysis questions	Thoughtful answers demonstrating understanding